

Metodologie di Programmazione (A-L)

II semestre - a.a. 2025 – 2026

Stream IV

a cura di Massimo La Morgia



SAPIENZA
UNIVERSITÀ DI ROMA

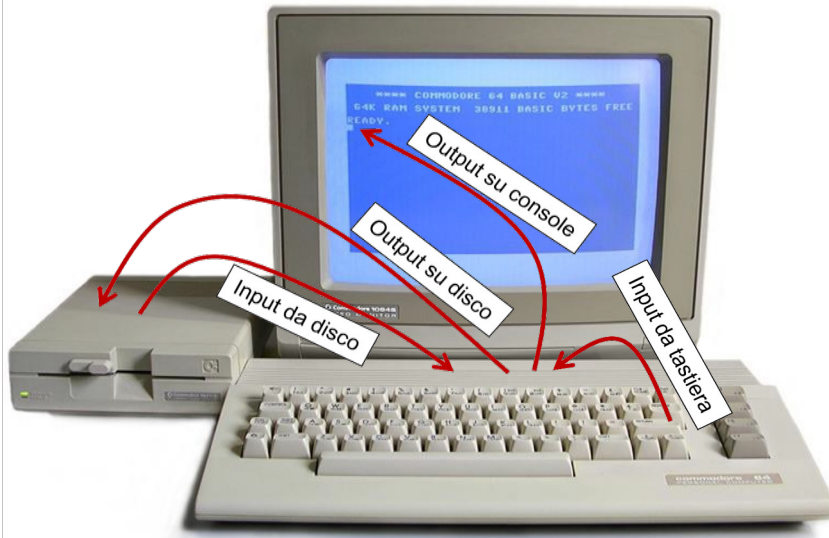
*Tutti i diritti relativi al presente materiale didattico ed al suo contenuto sono riservati a Sapienza e ai suoi autori (o docenti che lo hanno prodotto). È consentito l'uso personale dello stesso da parte dello studente a fini di studio. Ne è vietata nel modo più assoluto la diffusione, duplicazione, cessione, trasmissione, distribuzione a terzi o al pubblico pena le sanzioni applicabili per legge.

**I crediti sulle slide di questo corso sono riportati nell'ultima slide

Stream<T> parte IV

Input & Output

Stream<T>



Gli **Stream** di I/O sono un concetto ben distinto da **Stream API**.

Ottenere uno stream infinito

- L'interfaccia `Stream` espone un metodo `iterate` che, partendo dal primo argomento, restituisce uno stream infinito con valori successivi applicando la funzione passata come secondo argomento:

```
Stream<Integer> numbers =  
    Stream.iterate(0, n -> n+10);
```

- E' possibile limitare uno stream infinito con il metodo `limit`:

```
numbers.limit(5).forEach(System.out::println);
```

```
// 0, 10, 20, 30, 40
```

Implementare l'iterazione mediante stream

- Da Java 9, l'interfaccia `Stream` espone un secondo metodo `iterate` che, partendo dal primo argomento, restituisce uno stream di valori successivi applicando la funzione passata come terzo argomento e uscendo quando il predicato del secondo argomento è `false`:

`// 0, 10, 20, 30, 40`

`Stream<Integer> numbers = Stream`

`.iterate(0, n -> n < 50, n -> n+10 ); // Stream<Integer>`

L'ordine delle operazioni conta (1)

```
Stream.of("d2", "a2", "b1", "b3", "c")
    .map(s -> {
        System.out.println("map: " + s);
        return s.toUpperCase();
    })
    .filter(s -> {
        System.out.println("filter: " + s);
        return s.startsWith("A");
    })
    .forEach(s -> System.out.println("forEach: " + s));
```

Output:

```
map:d2
filter:D2
map:a2
filter: A2
map: b1
filter: B1
map: b3
filter: B3
map: c
filter: C
```

```
forEach: A2
```

L'ordine delle operazioni conta (1)

```
Stream.of("d2", "a2", "b1", "b3", "c")
    .map(s -> {
        System.out.println("map: " + s);
        return s.toUpperCase();
    })
    .filter(s -> {
        System.out.println("filter: " + s);
        return s.startsWith("A");
    })
    .forEach(s -> System.out.println("forEach: " + s));
```

Output:

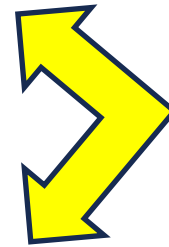
```
map:d2
filter:D2
map:a2
filter: A2
map: b1
filter: B1
map: b3
filter: B3
map: c
filter: C
```

forEach: A2

Notiamo che: tutte le
stringhe vengono mappate
anche se poi saranno filtrate!

L'ordine delle operazioni conta (2)

```
Stream.of("d2", "a2", "b1", "b3", "c")
    .filter(s -> {
        System.out.println("filter: " + s);
        return s.startsWith("a");
    })
    .map(s -> {
        System.out.println("map: " + s);
        return s.toUpperCase();
    })
    .forEach(s -> System.out.println("forEach: " + s));
```



Output:
filter: d2
filter:a2
map:a2
filter:b1
filter:b3
filter:c
forEach: A2

Output:
map:d2
filter:D2
map:a2
filter: A2
map: b1
filter: B1
map: b3
filter: B3
map: c
filter: C

forEach: A2

- Invertendo l'ordine di filter e map la pipeline è molto più veloce

L'ordine delle operazioni conta (3): sorted è stateful!

- Aggiungiamo un ordinamento dei valori nello stream:

```
Stream.of("d2", "a2", "b1", "b3", "c")
    .sorted((s1, s2) -> {
        System.out.printf("sort: %s; %s\n", s1, s2);
        return s1.compareTo(s2);
    })
    .filter(s -> {
        System.out.println("filter: " + s);
        return s.startsWith("a");
    })
    .map(s -> {
        System.out.println("map: " + s);
        return s.toUpperCase();
    }).forEach(s -> System.out.println("forEach: " + s));
```

Output:

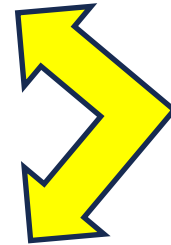
```
sort:  a2; d2
sort:  b1; a2
sort:  b1; d2
sort:  b1; a2
sort:  b3; b1
sort:  b3; d2
sort:  c; b3
sort:  c; d2
filter:a2
map:a2
filter:b1
filter:b3
filter: c
filter:d2

forEach:
A2
```

L'ordine delle operazioni conta (4): sorted è stateful!

- Modifichiamo l'ordine delle operazioni e ottimizziamo:

```
Stream.of("d2", "a2", "b1", "b3", "c")
    .filter(s -> { System.out.println("filter: " +
        s); return s.startsWith("a");
    })
    .sorted((s1, s2) -> {
        System.out.printf("sort: %s; %s\n", s1, s2); return
        s1.compareTo(s2);
    })
    .map(s -> {
        System.out.println("map: " + s);
        return s.toUpperCase();
    }).forEach(s -> System.out.println("forEach: " + s));
```



Output
filter: d2
filter: a2
filter: b1
filter: b3
filter: c
map: a2
forEach:A2

Output:
sort: a2; d2
sort: b1; a2
sort: b1; d2
sort: b1; a2
sort: b3; b1
sort: b3; d2
sort: c; b3
sort: c; d2
filter:a2
map:a2
filter:b1
filter:b3
filter: c
filter:d2

forEach:
A2

L'ordine di esecuzione delle operazioni è ottimizzato

```
Stream.of("domus", "aurea", "bologna", "burro", "charro")  
  .map(s -> {  
    System.out.println("map: " + s);  
    return s.toUpperCase();  
  })  
  .anyMatch(s -> {  
    System.out.println("anyMatch: " + s);  
    return s.startsWith("A");  
  });
```

Output:

map: domus

anyMatch: DOMUS

map: aurea

anyMatch: AUREA

L'ordine di esecuzione delle operazioni è ottimizzato

```
List<Integer> numbers = Arrays.asList(1, 2, 3, 4, 5, 6, 7, 8);
```

```
List<Integer> twoEvenSquares =  
    numbers.stream()  
        .filter(n -> {  
            System.out.println("filtering " + n);  
            return n % 2 == 0;  
        })  
        .map(n -> {  
            System.out.println("mapping " + n);  
            return n * n;  
        })  
        .limit(2)  
        .collect(toList());
```

Output:

```
filter 1  
filter 2  
map 2  
filter 3  
filter 4  
map 4
```

Fare copie di stream

- Abbiamo detto che gli stream non sono riutilizzabili
- Tuttavia è possibile creare un “costruttore” di stream mediante una lambda
- Ad es:

```
Supplier<Stream<String>> streamSupplier = () ->
```

```
    Stream.of("d2", "a2", "b1", "b3", "c")
```

```
        .filter(s -> s.startsWith("a"));
```

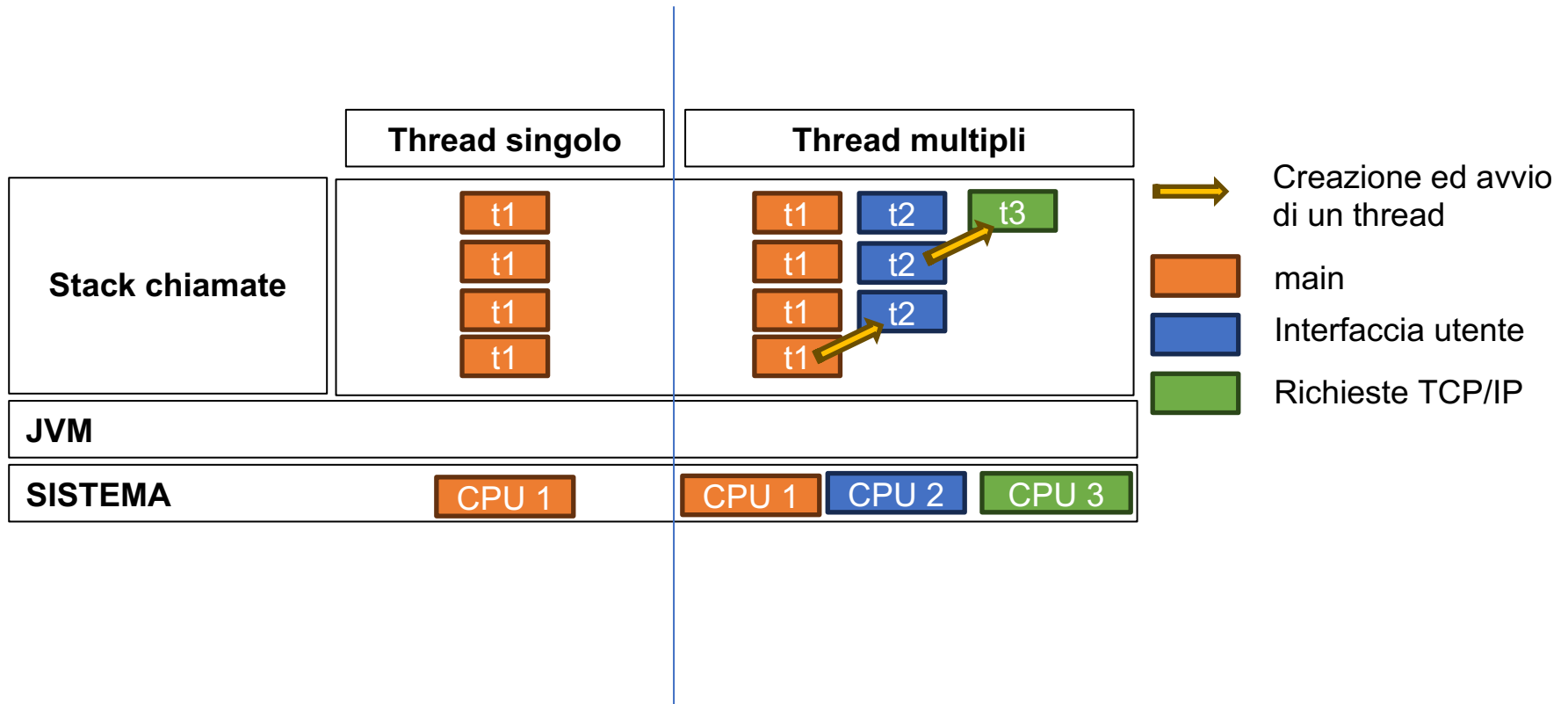
```
streamSupplier.get().anyMatch(s -> true); // true
```

```
streamSupplier.get().noneMatch(s -> true); // false
```

- Ovviamente ha più senso se, invece di un **Supplier**, abbiamo un metodo che prende in input una **Collection** e restituisce uno stream su tale collection

Single Thread e Multi Thread

- In Java si possono generare più rami dell'albero delle chiamate e i rami sono eseguiti in parallelo sulla base delle risorse di calcolo disponibili



Creare un Thread

- Possiamo utilizzare la Classe **Thread**

main Thread

```
...  
Thread thread = new Thread(  
    ()-> {/*codice da eseguire in parallelo */ }  
);  
thread.start();  
...
```

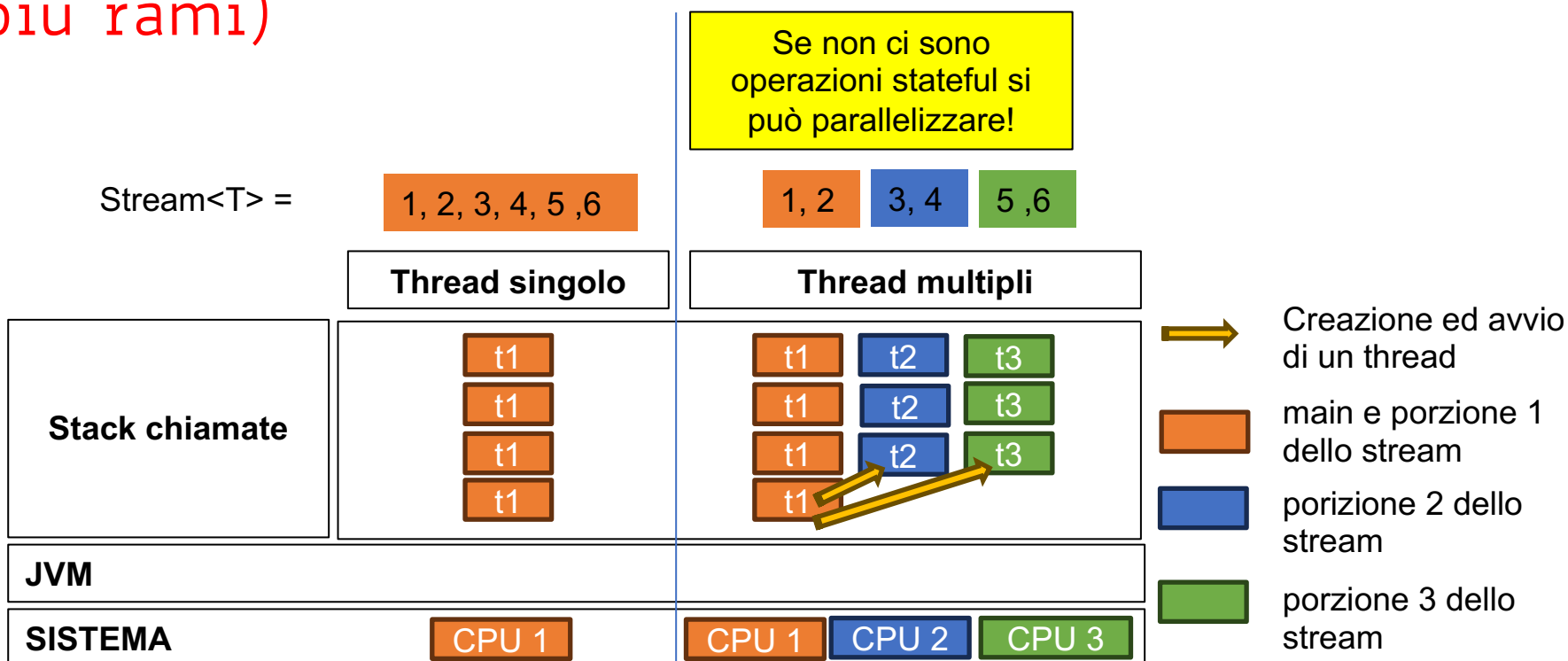
nuovo Thread

/*codice da eseguire in parallelo */

```
@FunctionalInterface  
public interface Runnable {  
    void run();  
}
```

Stream sequenziali e stream paralleli

- Le operazioni su stream sequenziali sono effettuate in un singolo thread (albero delle chiamate che consiste in un solo ramo)
- Le operazioni su stream paralleli, invece, sono effettuate concorrentemente su thread multipli (albero delle chiamate che consiste di più rami)



Perché creare uno stream parallelo?

Esempio:

```
int max = 1000000;
```

```
List<String> values = new ArrayList<>(max);
```

```
for (int i = 0; i < max; i++)
```

```
{
```

```
    UUID uuid = UUID.randomUUID();
```

```
    values.add(uuid.toString());
```

```
}
```

Perché creare uno stream parallelo?

Con l'ordinamento sequenziale:

```
long t0 = System.nanoTime();  
long count = values.stream().sorted().count();  
System.out.println(count);  
long t1 = System.nanoTime();  
long millis = TimeUnit.NANOSECONDS.toMillis(t1 - t0);  
System.out.printf("ordinamento sequenziale: %d ms", millis);  
// ordinamento sequenziale: 899 ms
```

Perché creare uno stream parallelo?

Con l'ordinamento parallelo:

```
long t0 = System.nanoTime();  
long count = values.parallelStream().sorted().count();  
System.out.println(count);  
long t1 = System.nanoTime();  
long millis = TimeUnit.NANOSECONDS.toMillis(t1 - t0);  
System.out.printf("ordinamento parallelo: %d ms", millis);  
  
// ordinamento parallelo: 472 ms
```

Uno stream parallelo può essere MOLTO PIU' LENTO di uno stream sequenziale

- Quando le operazioni sono bloccanti:

```
List<String> ls = IntStream.range(0, 100000).mapToObj(Integer::toString).collect(toList());
```

```
l.parallelStream/stream()
```

```
.filter(s -> {
```

```
    System.out.format("filter: %s [%s]\n", s, Thread.currentThread().getName());
```

```
    return true; })
```

```
.map(s -> {
```

```
    System.out.format("map: %s [%s]\n", s, Thread.currentThread().getName());
```

```
    return s.toUpperCase(); })
```

```
.sorted((s1, s2) -> {
```

```
    System.out.format("sort: %s <> %s [%s]\n", s1, s2,
```

```
    Thread.currentThread().getName());
```

```
    return s1.compareTo(s2); })
```

```
.forEach(s -> System.out.format("forEach: %s [%s]\n", s, Thread.currentThread().  
                                getName()));
```

parallelStream:

87474402 ns

stream:

56447233 ns

Quando usare uno stream parallelo?

- Quando il problema è parallelizzabile
- Quando posso permettermi di usare più risorse (es. tutti i core del processore)
- La dimensione del problema è tale da giustificare il sovraccarico dovuto alla parallelizzazione

Le mappe in Java 8 e 9

- Le mappe non supportano gli stream
- Ma forniscono numerose operazioni aggiuntive in Java 8

```
Map<Integer, String> map = new HashMap<>();  
for (int i = 0; i < 10; i++) map.putIfAbsent(i, "val" + i);  
map.forEach((id, val) -> System.out.println(val));
```

- In Java 9, abbiamo `Map.of` per creare una mappa immutabile

Le mappe in Java 8

Se l'elemento con chiave 3 è presente, modifica il valore associatogli utilizzando la **BiFunction** in input come secondo parametro:

```
map.computeIfPresent(3, (key, val) -> val + key);
```

```
map.get(3); // nuovo valore
```

Se l'elemento con chiave 23 non è presente, aggiungi la chiave 23 e associa il valore restituito dalla **Function**:

```
map.computeIfAbsent(23, key -> "val" + key);
```

```
map.containsKey(23); // true
```

Se l'elemento con chiave 3 non è presente o il valore associato è null, aggiungi la chiave 3 e associa il valore restituito dalla **Function**:

```
map.computeIfAbsent(3, key -> "bam");
```

```
map.get(3); // restituisce "bam"
```

Le mappe in Java 8

```
map.remove(3, "val33"); // removes the pair
```

```
map.get(3); // null
```

```
map.getDefault(3, "not found"); // not found
```

```
map.merge(9, "val9", (value, newValue) ->  
    value.concat(newValue)  
    );
```

```
map.get(9); // valorePrecedenteSeEsistenete+"val9"
```


API per data e ora

- Package `java.time`
- `ZoneId`: rappresenta un fuso orario
- `LocalTime`: rappresenta un'ora locale
- `LocalDate`: rappresenta una data locale
- `LocalDateTime`: rappresenta una data e ora
- `Clock`: accede all'attuale data e ora

```
Clock clock = Clock.systemDefaultZone();
```

```
long millis = clock.millis();
```

Restituisce l'attuale data e ora in millisecondi,
equivalente a `System.currentTimeMillis()`

Esempio: gestire la data

- `LocalDate today = LocalDate.now();`
- Alcuni metodi:

`getDayOfMonth() : int - LocalDate`

`getDayOfWeek() : DayOfWeek - LocalDate`

`getDayOfYear() : int - LocalDate`

`getEra() : Era - LocalDate`

`getLong(TemporalField arg0) : long - LocalDate`

`getMonth() : Month - LocalDate`

`getMonthValue() : int - LocalDate`

`getYear() : int - LocalDate`



Esercizio

- Sia data la seguente classe:

```
public class Titolo
{
    public enum Allineamento { CX, SX, DX }
    private Allineamento allineamento;
    private List<Riga> righe;

    public Titolo(Allineamento a) { this(a, new ArrayList<>()); }
    public Titolo(Allineamento a, List<Riga> righe) { allineamento = a; this.righe = righe; }
    public void add(Riga r) { righe.add(r); }
    public boolean isCentered() { return allineamento == Allineamento.CX; }
    @Override public String toString() { return righe.toString(); }
    public Allineamento getAllineamento() { return allineamento; }
    public List<Riga> getRighe() { return new ArrayList<>(righe); }

    static public class Riga
    {
        private String riga;
        private int numero;

        public Riga(String riga, int numero) { this.riga = riga; this.numero = numero; }
        public Riga(String riga) { this(riga, -1); }
        @Override public String toString() { return (numero == -1 ? "" : numero+": ")+riga; }
    }
}
```

Utilizzare gli stream per ottenere i seguenti dati (svolgere ciascun punto sia con riferimenti a metodi che lambda)

- A partire da una `List<Titolo>`, si calcolino **senza precalcoli di strutture dati intermedie** (ovvero, su una sola riga):
 1. l'insieme dei primi 5 titoli aventi al più una riga
 2. la lista dei soli titoli centrati e in ordine alfabetico
 3. mappa da allineamento a lista di titoli
 4. mappa da allineamento a insieme di titoli
 5. mappa da allineamento alla concatenazione delle stringhe dei titoli
 6. mappa da allineamento alla lista delle stringhe dei titoli
 7. insieme delle righe dei titoli ciascuna rappresentata sotto forma di stringa
 8. mappa dei conteggi delle parole nelle righe di titoli calcolate al punto precedente (questo esercizio richiede l'utilizzo di `flatMap`)
 - Utilizzando il `Collectors.groupingBy`
 - Utilizzando il `Collectors.toMap`



Credits

Le slide di questo corso sono basate sulle slide del Prof. Faralli che sono a loro volta una rielaborazione delle slide del Prof. Navigli

In aggiunta, le slide sono state revisionate dagli studenti borsisti della Facoltà di Ingegneria Informatica, Informatica e Statistica: Mario Marra e Paolo Straniero.

Font ad alta leggibilità Biancoenero® di biancoenero edizioni srl, disegnata da Umberto Mischi, con la consulenza di Alessandra Finzi, Daniele Zanoni e Luciano Perondi (Brevetto n. RM20110000128).

Disponibile gratuitamente per tutte le istituzioni e i privati che la utilizzino per scopi non commerciali.